

Architecting AI Cost Governance: Guide to monitor Gemini Token Consumption on Google Cloud

– github/mandarvs

– June 2026

-v4

Executive Summary: This technical guide provides you with a step-by-step methodology for establishing deep visibility into token consumption and billing, implementing and configuring threshold-based alerting for Vertex AI Gemini models using Google Cloud Monitoring and Logging services and cost visibility using Google Cloud Billing data and reports.

With the rapid deployment of agentic systems and complex LLM-powered workflows, organizations are facing unprecedented scrutiny regarding the Return on Investment (RoI) of their artificial intelligence spend. Because large language models are primarily billed based on token throughput, establishing a mature cost governance framework is no longer optional; it is a critical operational requirement to prevent budget overruns and ensure fiscal accountability.

The document is in **two sections**, the first section details how you can monitor almost real time **monitoring of token consumption** and the second section details how you can use **Google Cloud billing data to monitor token costs**. Billing data is usually updated in batches and shows the final landed cost for GCP resources used.

.

Section 1 — Analyzing Token Consumption Dynamics by Model

Part 1 - Configuring metrics

This implementation is designed to provide definitive answers to three primary business questions:

1. **Quantifying Aggregate Usage:** What is our total token consumption, categorized by model type and temporal frequency?
2. **User Attribution:** Which specific developers, service accounts, or end-users are responsible for driving this consumption?
3. **Proactive Governance:** How can we implement automated alerting mechanisms to notify stakeholders before consumption exceeds pre-defined budgetary thresholds?

Step 1.1: Accessing the Monitoring Metrics Explorer

Google Cloud Console → Monitoring → Metrics Explorer

Step 1.2: Identifying the Vertex AI Token Telemetry

- **Metric:** Vertex AI Model Garden Publisher Model → Token count
- Full metric ID:
`aiplatform.googleapis.com/publisher/online_serving/token_count`
- Resource type: `aiplatform.googleapis.com/PublisherModel`

Step 1.3: Defining the Temporal Reporting Window

Top-right of Metrics Explorer → change to **Last 14 days** (or whatever range matters for your reporting).

Step 1.4: Configuring Multi-Dimensional Aggregation

- **Aggregation:** Sum
- **by:** type (splits input vs output tokens)
- **Min interval:** 1d (daily buckets)

Strategic Rationale for Daily Buckets: Financial planning and cloud FinOps teams typically operate on daily or monthly cadences. While minute-by-minute data is useful for debugging, daily buckets provide the optimal signal-to-noise ratio for identifying cost trends and significant anomalies without the overhead of operational volatility.

Step 1.5: Understanding the Delta Metric Normalization

You may notice the Y-axis displaying `tokens/s` instead of absolute totals. This occurs because `token_count` is a **DELTA metric**. By default, Cloud Monitoring applies an `ALIGN_RATE` function, which normalizes the data over the alignment period. To view the raw, accumulated total, you must bypass the standard UI builder.

Step 1.6: Utilizing PromQL for Precise Daily Accumulation

Click `</>` PromQL (top-right of the query row) and paste:

None

```
sum by (model_user_id, type) (  
  increase(  
    {  
      "__name__"="aiplatform.googleapis.com/publisher/online_serving/token_count",  
      "monitored_resource"="aiplatform.googleapis.com/PublisherModel",  
      "model_user_id"=~"gemini.*"  
    }[1d]  
  )  
)
```

- `increase(...[1d])` returns total tokens accumulated over the trailing 24h window
- `sum by (model_user_id, type)` groups by model and input/output type
- The regex `gemini.*` keeps the chart focused on Gemini family models (drop it to include Imagen, embeddings, etc.)

Step 1.7: Optimizing Visualization through Widget Selection

Right-side **Display** panel → **Widget type** → **Stacked bar chart**.

You now have one bar per day, stacked by `(model, input/output)`.

Step 1.8: Validating Data Accuracy

Cross-check with hourly buckets:

None

```
# Should match the daily query above (within <0.1% extrapolation noise)  
sum_over_time(  
  sum by (model_user_id, type) (  
    increase({...}[1h])  
  )[1d:1h]  
)
```

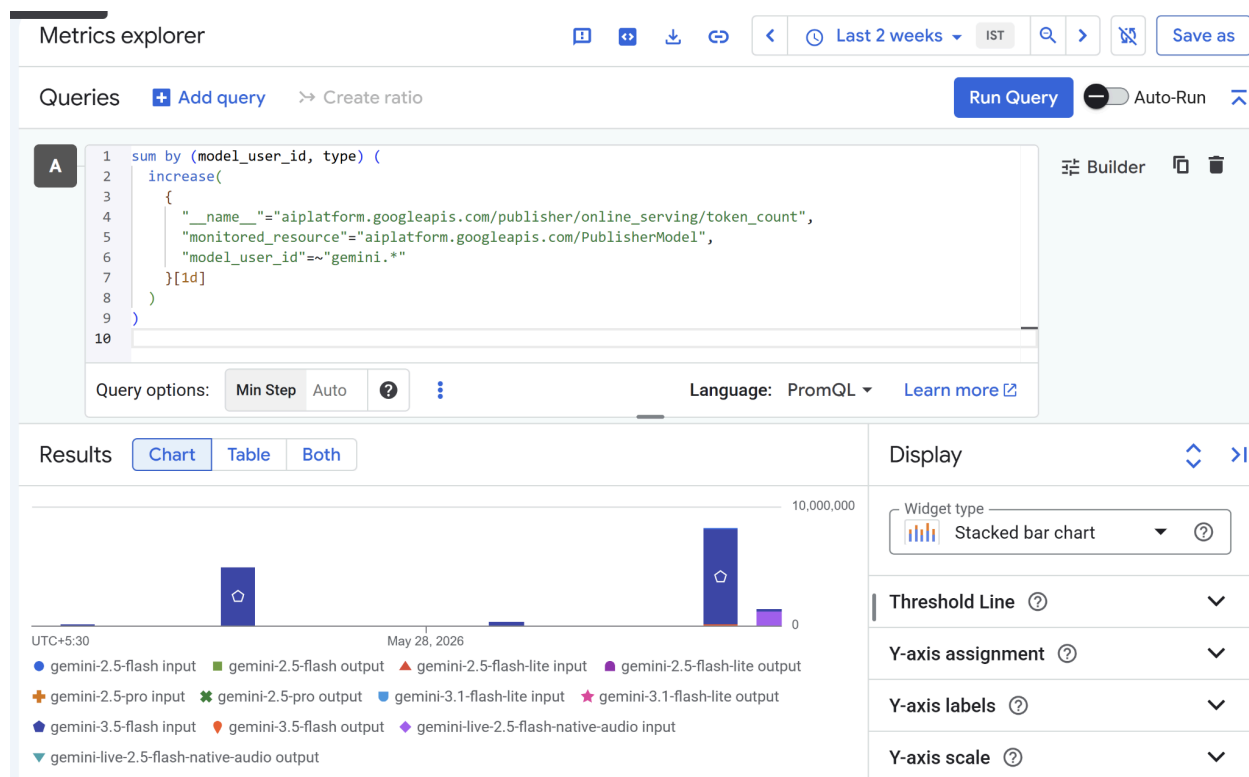


Fig: Metrics explorer showing aggregate token consumption by model and “type”

Analytical Outcomes of Part 1

What this gives you	Why it matters
Daily token totals per model	First step is always visibility. Until usage is measured, it cannot be governed.
Input vs output split	Output tokens are typically 3–5× more expensive than input tokens. Knowing the ratio helps target optimizations (caching, prompt compression).
Per-model breakdown	Identifies workloads that could move to cheaper tiers (gemini-2.5-flash vs gemini-2.5-pro) without quality loss.

What this gives you	Why it matters
Trailing 24h, not calendar day	The [1d] window is a sliding 24h. For exact midnight-to-midnight reporting, export to BigQuery (see Section 2).

Actionable Business Intelligence: This visibility enables you to make data-driven architectural decisions. For instance, if your data reveals that 70% of spend is allocated to the Pro model for tasks that do not require high reasoning capabilities, you can mandate a migration to the Flash model, achieving significant cost savings with negligible impact on quality.

Part 2 — Implementing Granular Per-User Attribution via Cloud Audit Logs

Addressing the Metadata Limitation

While the `token_count` metric provides volume, it lacks the **caller identity** context necessary for chargeback. To bridge this gap, you must integrate **Cloud Audit Logs**, which record the principal email associated with every API request made to the Vertex AI service.

Step 2.1: Enable Data Access audit logs for Vertex AI

Navigation: IAM & Admin → Audit Logs → Vertex AI API → Enable 'Data Read' → Save

Resource Impact Analysis: Data Access logs contribute to ingestion billing. While the volume is typically low for direct human interaction, high-throughput automated systems can generate significant log data. It is recommended to monitor the logging spend alongside the API spend.

Step 2.2: Verify audit logs are being generated

In **Logs Explorer**, paste:

```
None
logName=~"cloudaudit.googleapis.com%2Fdata_access"
protoPayload.serviceName="aiplatform.googleapis.com"
protoPayload.methodName=~".*GenerateContent.*"
```

Each entry contains:

- `protoPayload.authenticationInfo.principalEmail` — the calling identity
- `protoPayload.resourceName` — includes the model
- `timestamp`

Step 2.3: Engineering a Custom Log-Based Metric

Logging → Log-based Metrics → Create Metric

- **Type:** Counter
- **Name:** `gemini_requests_by_user` (lowercase, snake_case — the name is locked once data flows)
- **Filter:** the query from Step 2.2
- **Labels:**
 - `principal_email` ← `protoPayload.authenticationInfo.principalEmail` (STRING)
 - `model` ← `protoPayload.request.model` (STRING) — fallback: parse from `resourceName`

Step 2.4: Wait for data and find it in Metrics Explorer

Note that log-based metrics are **not retroactive**; they begin aggregating data only from the moment of creation. Ensure you trigger test requests to populate the metric series before attempting to build dashboards.

Trigger a few Gemini CLI calls, wait ~1–2 minutes, then in Metrics Explorer:

- Search for `gemini_requests_by_user`, OR
- Use the `:` menu → **View in Metrics Explorer** from the log-based metric page

Step 2.5: Chart it with PromQL

```
None
sum by (principal_email, model) (
  increase(
    {
      "__name__"="logging.googleapis.com/user/gemini_requests_by_user",
      "monitored_resource"="audited_resource"
    }[1d]
  )
)
```

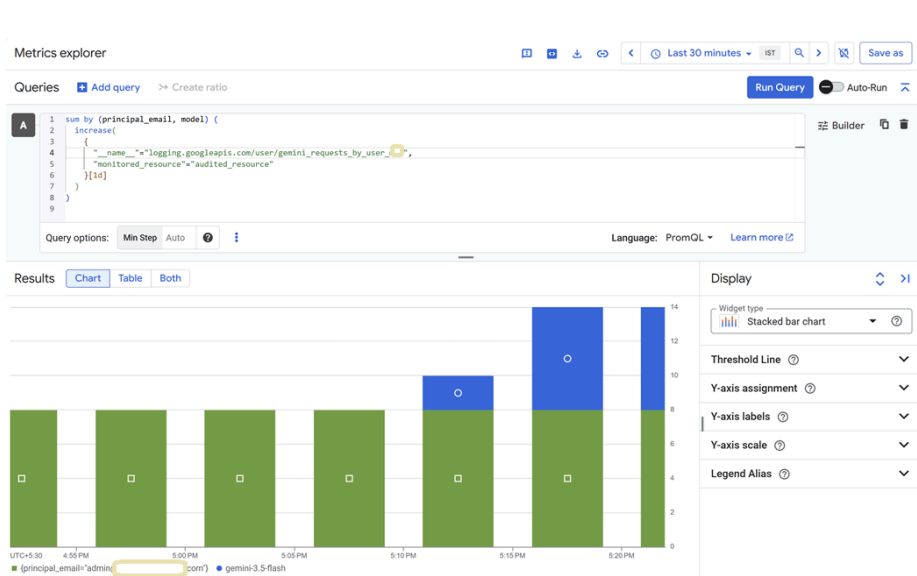


Fig : Custom metric tracking the request count by user.

Caveat — who shows up

Understanding the identity resolution logic:

Calling pattern	What principalEmail will be
End users running Gemini CLI with gcloud auth login or antigravity	The actual user (user@yourdomain.com)
Backend service calling Vertex AI on behalf of users	The service account (end-user identity is invisible)
OAuth-authenticated web app	The OAuth-authenticated user
Notebook / Workbench	The notebook owner

If your end users go through a backend, audit logs will only show the service account — you must instrument the application layer to capture true end-user identity (log a custom field with each request).

Analytical Outcomes of Part 2

What this gives you	Why it matters
Per-user request count	Identifies power users and outlier behavior. Sufficient for operational visibility.
Not per-user token count	Audit logs do not capture tokens. For precise chargeback, you must either pro-rate by request count or instrument the application.
Service account opacity	If a backend service is in front, audit logs show only the service account. End-user attribution requires application-level logging.

Strategic Benefit: By identifying specific users or applications with outlier consumption patterns, you can proactively offer training on prompt engineering or optimize specific backend modules that are unintentionally generating excessive request volumes.

Part 3 — Constructing Automated Alerting for Budgetary Compliance

Step 3.1: Configuring Communication Channels

[Monitoring](#) → [Alerting](#) → [Edit Notification Channels](#) → add Email / Slack / PagerDuty / webhook.

Step 3.2: Initializing the Alert Policy Framework

[Monitoring](#) → [Alerting](#) → [+ Create Policy](#)

Step 3.3: Scripting the Threshold Condition

Switch to **PromQL** mode. **The threshold goes inside the query** in the new UI — alerts fire when the expression returns a non-empty result:

```
None
sum(
  increase(
    {
```



```

    "__name__"="aiplatform.googleapis.com/publisher/online_serving/token_count",
    "monitored_resource"="aiplatform.googleapis.com/PublisherModel",
    "model_user_id"=~"gemini.*"
  }[1d]
)
) > 15000000

```

Replace **15000000** with your actual daily budget threshold. The chart preview will be empty when the condition is false (no alert) and show a line only when breached.

Step 3.4: Optimizing Trigger Sensitivity

Field	Recommended value	Why
Retest window	300 sec (5 min)	Condition must hold for 5 minutes before firing — avoids one-off spikes
Evaluation interval	60 second(s)	How often the query runs. 60s halves alerting cost vs 30s once pricing kicks in (Sept 2026)
Condition name	Gemini daily token budget exceeded	Shows in incident emails

Step 3.5: Validation and Functional Testing

Temporarily lower the threshold (e.g. **> 5000000**) to confirm the alert fires. The chart preview will immediately show data when the condition is true. Then restore your real threshold.

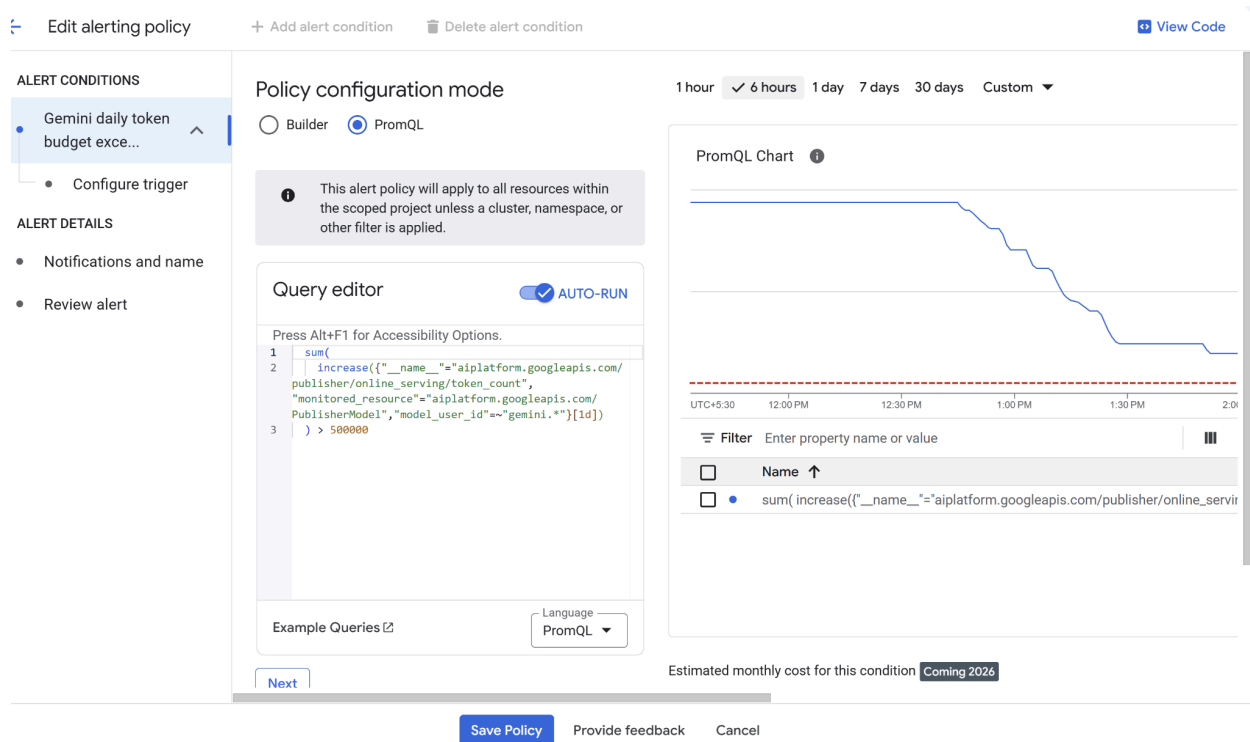


Fig: Alert Policy config screen

Step 3.6: Enriching Incident Notifications

- Add notification channel(s)
- **Auto-close duration:** 1 hour (incident resolves when trailing 24h drops below threshold)
- **Incident description** (Markdown shown in alert emails):

None

Daily Gemini token consumption (trailing 24h) exceeded \$threshold tokens.

Investigate top consumers via the `gemini\_requests\_by\_user` log-based metric.
Investigate model mix in Metrics Explorer with the per-model PromQL query.

Advanced Configuration — Per-Model Isolation

To alert when *any single model* exceeds the threshold (rather than total):

None

```
sum by (model_user_id) (
  increase({...}[1d])
) > 5000000
```

The alert fires one incident per breaching model, with `model_user_id` in the labels — useful for routing different models to different teams.

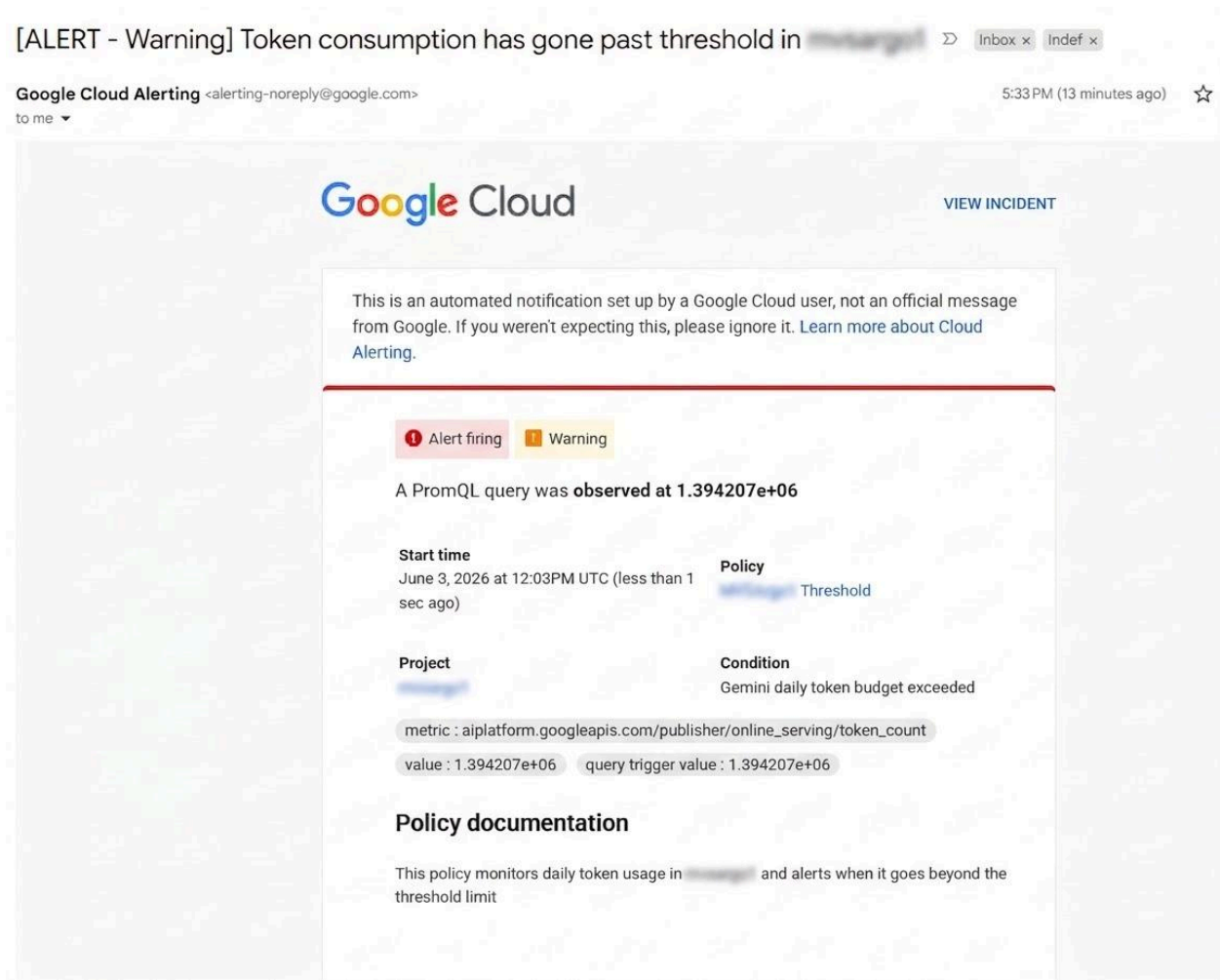


Fig: Sample email alert of a threshold breach

Governance Strategic Value — Part 3

What this gives you	Why it matters
Pre-bill detection	Catches runaway usage <i>before</i> the monthly invoice arrives. The alternative — discovering a 10× spike in Cloud Billing 30 days later — is much more expensive.

What this gives you	Why it matters
Threshold tuning forces a conversation	Picking the number requires Engineering and Finance to agree on a daily budget. That conversation is itself the governance win.
Per-model alerts enable targeted action	Generic "total budget exceeded" forces a fire drill across all workloads. <code>gemini-2.5-pro budget exceeded</code> points directly at the workload to investigate.
Trailing 24h = continuous evaluation	The alert can stay open for ~24h after consumption stops (waiting for the rolling window to clear). For a "fire once per day" report-style alert, use Cloud Scheduler + Cloud Function instead.

Core Objective: Your primary goal is to shift from a reactive "invoice-driven" posture to a proactive "telemetry-driven" posture. This ensures that your technical teams are the first to know about spikes, allowing you to remediate within hours rather than weeks.

Part 4 — Optimal Resource Hierarchy for Team-Based Governance

To move beyond project-level visibility toward organizational governance, establishing a formal resource hierarchy is essential. The leaf level projects are dedicated to workloads which are expected to consume a high number of tokens viz. Customer or agent facing agents deployed to Gemini Enterprise Agent Platform, developers using Antigravity/gemini cli, workloads executing large AI driven workflows and those generating images and videos at scale.

The representative structure below ensures that Gemini API consumption (via Antigravity as an example) remains observable and manageable at scale.

- The Hierarchy:** The architecture begins with a top-level parent folder dedicated to "Development" or "AI Initiatives." Below this parent, sub-folders are organized by business unit or department (e.g., 'Lending', 'HR'). Each sub-folder finally houses dedicated GCP projects for individual application teams (e.g., a 'FarmloansApp' project within the 'Lending' sub-folder).
- Team Allotment:** Dedicated projects are assigned to specific development teams. This decentralization allows teams to manage their own service accounts, API keys, and model resources independently without cross-team interference.
- Strategic Benefits:**

- **Granular Token Monitoring:** Enables monitoring and threshold-based alerts to be configured at the project level, revealing precisely which team or application is driving token consumption.
- **Fiscal Accountability:** Directly facilitates internal chargeback models by attributing AI spend to the specific business units and applications that generated it.
- **Operational Isolation:** Ensures that high-throughput experimentation from one team does not impact the stability of another team's workloads.

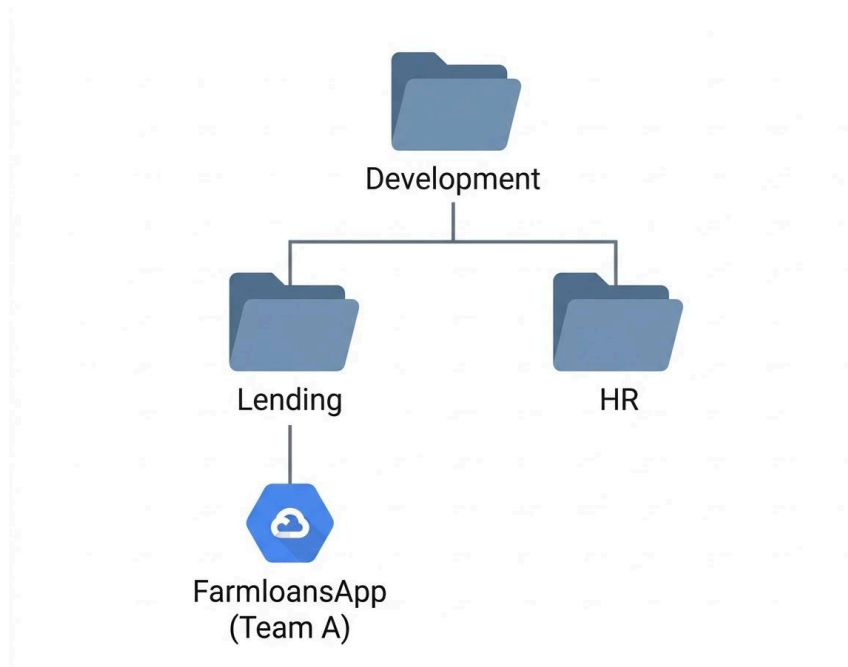


Fig: Recommended GCP Resource Hierarchy for Anti-gravity Governance

Gemini Request API labelling:

Google Cloud Gemini APIs allow the client to tag custom metadata to requests in the form of request labels which are key value pairs viz. “agent”:”hr_agent_v1” , “department”:hr”, the label keys are decided by user and are made available in the detail billing data exports and can be used to further analyze cost along business decided dimensions. More details on custom metadata labels [here](#). This feature is currently not directly available in antigravity cli.

Part 5— Moving to Mature Cost Governance

As your organization's AI maturity increases, you may require even more granular data. This involves moving beyond simple monitoring and into the realm of data warehousing and business intelligence by syncing audit and billing data into BigQuery.

Capability	Tool	When to add it
Per-day, per-user, per-model token breakdown (not request count)	BigQuery sink of audit logs + Monitoring metric, joined on time bucket	When chargeback accuracy matters
Cost dollars (not tokens)	Cloud Billing → Reports, filter by Vertex AI SKUs	Always — pair with this for the financial view
Per-user precise token attribution	Application-level logging of <code>usage_metadata</code> from each Gemini response	When per-user chargeback is a hard requirement
Long-term trend analysis	Export to BigQuery + Looker Studio	When you need quarter-over-quarter comparisons
Remediations: Optimizing tools	Finetune “skills” used by agents and improve promoting techniques	Implement aggressive tuning + developer training to limit token usage.
Remediations: Shift to price performant models	Antigravity allows users to switch models being used. ‘Flash’ models should be preferred unless “pro” models generate more optimal code	By default everyone uses “Flash” models viz. Gemini-3.5-flash

Comparative Analysis: Governance Posture Transformation

Dimension	Before this setup	After this setup
Visibility latency	30 days (next invoice)	Real-time (chart) / 5 min (alert)
Granularity	Project-level cost only	Per model, per day, per token type
Identity attribution	None	Per-user request counts (via audit logs)
Cost-spike detection	Discovered in monthly billing review	Alerted within minutes of breach
Optimization signal	"Bill is too high"	"Model X drove 60% of yesterday's tokens — investigate"
Chargeback capability	None	Per-user request count (proportional)
Governance posture	Reactive	Proactive

The most significant outcome of implementing this stack is the **cultural shift in cost accountability**. By moving the financial conversation upstream to the engineering level and providing transparent, real-time data, you can foster a sustainable and efficient AI adoption strategy.

Section 2 : Monitoring token consumption costs

Section 1 discusses real-time monitoring of token consumption; to find out the “cost” incurred, you have to look at cloud billing data. The section below discusses how you can use cloud billing reports available in the GCP console as well as drilling down deeper by analyzing the billing data exported to Google BigQuery.

Inbuilt GCP billing reports

Google cloud console provides multiple features to enable deep visibility into cloud costs. Best practices on GCP cost management are documented [here](#) and is a good place to start. The following section describes how to use the inbuilt billing report Reports functionality to monitor costs. Do note that cost data might take up to 24 hrs to be updated. Documentation on how to use Cloud Billing Reports is available [here](#)

Inbuilt reports can be accessed by navigating to the *Billing>Cost management>Reports* on the console. You can filter the report data to SKUs which relate to LLM token consumption and desired date range . Finding the right SKUs to filter on to narrow down on token consumption is a bit tedious, however you can use the text search filter to narrow down on the SKU of interest as shown in the screenshot below. A comprehensive list of Google Cloud platform SKUs is available [here](#)

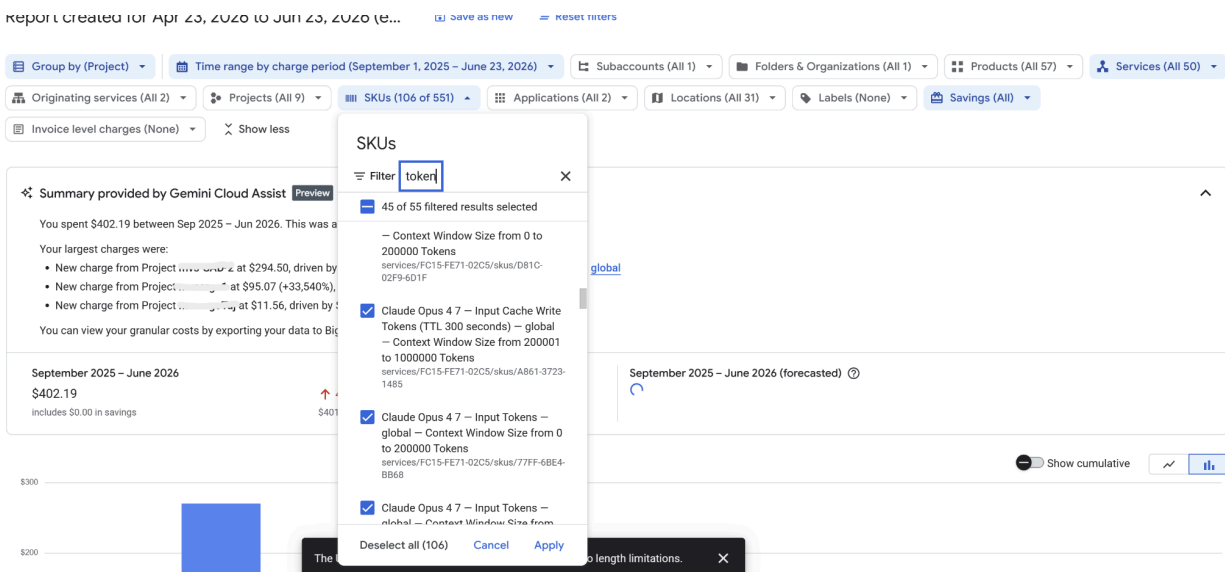


Fig: Filtering a billing report to token related SKUs

The report also provides an AI generated summary of the spend, generated by Gemini Cloud Assist, of your spends and chart visualization.

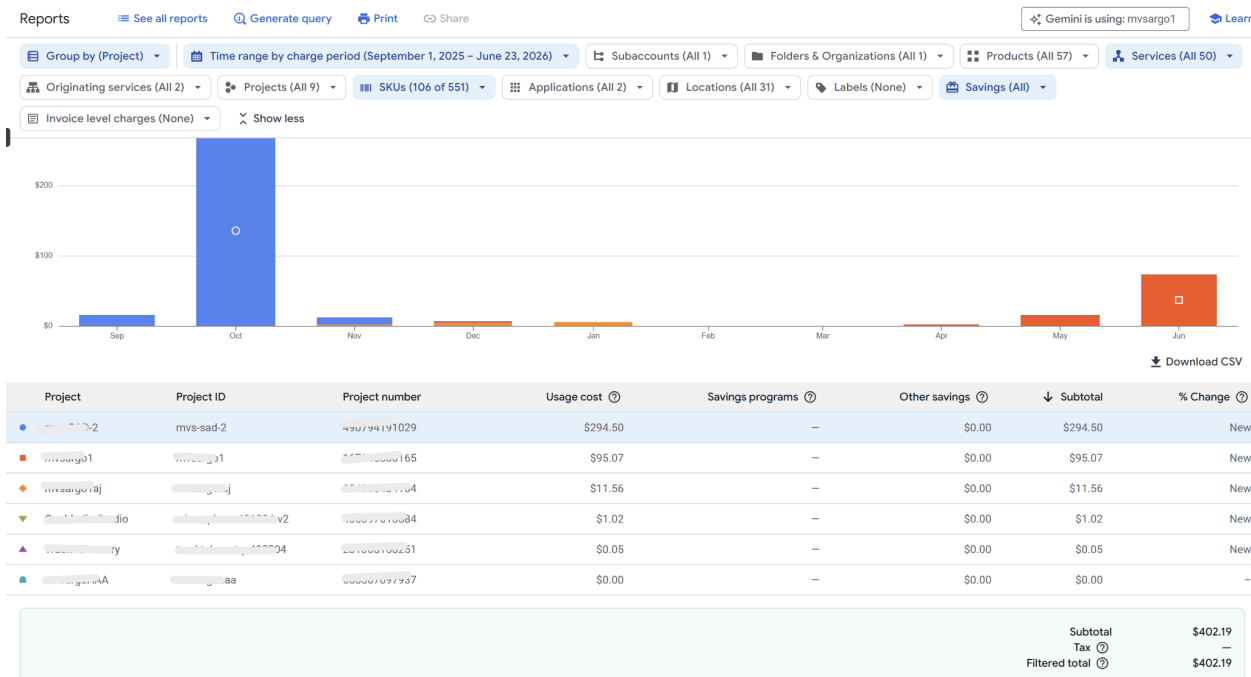


Fig: Chart and tabular output of the token costs

ADK Agent cost filter

For AI agents built with Google [ADK](#), it automatically injects a billing “label” key called “`adk_agent_name`” with value set to the agent name with all calls to Gemini APIs, which can be used to extract agent specific billing information and usually relates directly to token consumption. Please note that these labels can be overridden at any time with a custom set to achieve more granular visibility into your token billing consumption.

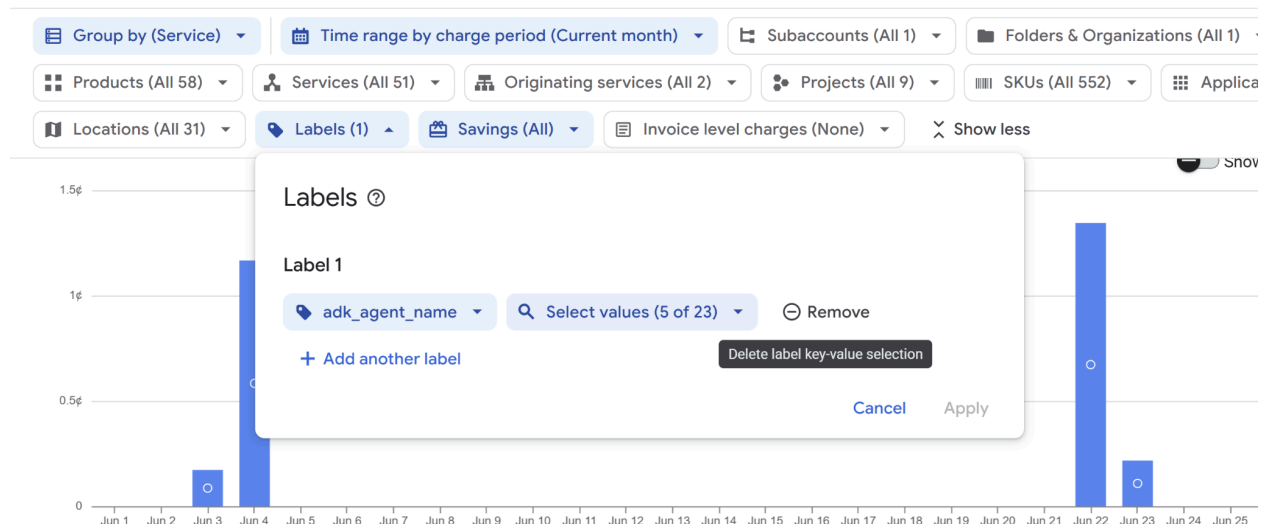


Fig: Filtering costs of ADK agents invoking Gemini API

Inbuilt cloud billing reports can be a great starting point for token cost analysis, however if deeper drill down capabilities are needed GCP supports exporting the billing data to a BigQuery (BQ) Dataset. The section below describes setting this up and then using **DataStudio** (free BI tool), **Connected Google Sheets** (for those who like to work with sheets) and **Chat agent** (using BQ Conversational Analytics API) to explore the billing data. Do note the steps below need some background in writing SQL queries and using BI tools.

The first step is to set up the Billing data export to BQ, if it is not set up already. The process is simple and documented [here](#). This will allow org-wide visibility into Gemini API and Vertex AI token consumption and spend, broken down by project, model, and input/output direction. Once the data gets populated in BQ, you will create a BigQuery view (**v_gemini_tokens**) which limits the required token data that any downstream dashboard (Looker Studio, Cloud Monitoring, custom) can query.

Gemini Token Billing Setup

Prerequisites

Requirement	Notes
Billing Account Administrator role	On the org's billing account. Needed to enable export.

Requirement	Notes
Central "ops" project for the export	Recommended single GCP project — e.g., <code>billAnalytics</code> . The export captures all projects on the billing account regardless of which project hosts the dataset.
<code>roles/bigquery.admin</code> on the ops project	To create the dataset and view.
<code>roles/bigquery.dataViewer</code> for dashboard users	Granted later, on the dataset only.

Part 1 — Enable Detailed Billing Export to BigQuery

The standard export gives daily SKU-level cost. The **detailed** export adds resource-level breakdowns and is what we use here.

1. Open **Billing** → **Billing Export** → **BigQuery export** in the GCP Console.
2. Under **Detailed usage cost**, click **Enable detailed Export**.
3. Choose the project that will host the dataset (e.g., `billAnalytics`).
4. Select an existing dataset or create a new one (e.g., `BillingDS`)
5. Click **Save**.

Verify the export

Export starts within a few hours. Verify with:

```
SQL
SELECT MIN(usage_start_time) AS first_row,
       MAX(usage_start_time) AS latest_row,
       COUNT(*) AS row_count
FROM `<PROJECT>.<DATASET>.gcp_billing_export_resource_v1_<BILLING_ID_SUFFIX>`;
```

The exact table name carries the billing account suffix; list with `bq ls <PROJECT> : <DATASET>` if unsure.

Part 2 — Create the Parsing View

Billing rows for Gemini have unstructured SKU descriptions. The view below normalizes them so downstream queries can group by clean `project_id` × `model_name` × `token_direction` keys.

Note : This query filters the results using the as-of-date first party models and will need tweaking as new models are announced OR to extend it to other models offered in a Model As A Service (MAAS) mode viz. Models from Anthropic

Replace the table identifier (<PROJECT>.<DATASET>) with yours before running.

SQL

```
CREATE OR REPLACE VIEW `<PROJECT>.<DATASET>.v_gemini_tokens` AS
WITH base AS (
  SELECT
    DATE(usage_start_time) AS usage_date,
    IFNULL(project.id, 'org-level (unattributed)') AS project_id,
    IFNULL(project.name, 'org-level (unattributed)') AS project_name,
    service.description AS service,
    TRIM(sku.description) AS raw_sku,
    sku.id AS sku_id,
    usage.amount AS usage_amount,
    usage.unit AS usage_unit,
    cost,
    cost + IFNULL((SELECT SUM(c.amount) FROM UNNEST(credits) c), 0) AS net_cost,
    currency,
    REGEXP_EXTRACT(LOWER(sku.description), r'gemini\s*(\d+(?:\.\d+)?)' ) AS
version_raw,
    REGEXP_EXTRACT(LOWER(sku.description),
r'gemini\s*\d+(?:\.\d+)?\s*(pro|flash)' ) AS tier,
    REGEXP_EXTRACT(LOWER(sku.description),
r'gemini\s*\d+(?:\.\d+)?\s*(?:pro|flash)\s*(lite|live)' ) AS subtier,
    _PARTITIONTIME AS partition_time,
    labels
  FROM `<PROJECT>.<DATASET>.gcp_billing_export_resource_v1_<BILLING_ID_SUFFIX>`
  WHERE
    (service.description = 'Gemini API' AND LOWER(sku.description) LIKE '%token
count%')
    OR (
      service.description = 'Vertex AI'
      AND REGEXP_CONTAINS(LOWER(sku.description), r'gemini\s*\d')
      AND (LOWER(sku.description) LIKE '%input%' OR LOWER(sku.description) LIKE
'%output%')
      AND (LOWER(sku.description) LIKE '%prediction%' OR LOWER(sku.description)
LIKE '%caching%')
```

```

        AND LOWER(sku.description) NOT LIKE '%model garden%'
    )
    OR (
        service.description = 'Vertex AI'
        AND (LOWER(sku.description) LIKE '%imagen%' OR LOWER(sku.description) LIKE
'%veo%')
    )
)
SELECT
    usage_date,
    project_id,
    project_name,
    CASE
        WHEN service = 'Gemini API' THEN 'Gemini API (AI Studio)'
        WHEN service = 'Vertex AI' THEN 'Vertex AI'
        ELSE service
    END AS surface,
    CASE
        WHEN LOWER(raw_sku) LIKE '%imagen%' THEN 'image-gen'
        WHEN LOWER(raw_sku) LIKE '%veo%' THEN 'video-gen'
        ELSE 'llm'
    END AS category,
    CASE
        WHEN LOWER(raw_sku) LIKE '%imagen%' THEN
            CONCAT('Imagen',
                IFNULL(CONCAT(' ', REGEXP_EXTRACT(raw_sku, r'(?i)imagen\s*(\d+)')), ''),
                IF(LOWER(raw_sku) LIKE '%fast%', ' Fast', ''))
        WHEN LOWER(raw_sku) LIKE '%veo%' THEN
            CONCAT('Veo ', REGEXP_EXTRACT(raw_sku, r'(?i)veo\s*(\d+)'),
                IF(LOWER(raw_sku) LIKE '%fast%', ' Fast', ''))
        WHEN tier IS NULL THEN CONCAT('UNPARSED: ', raw_sku)
        ELSE CONCAT(
            'Gemini ',
            IF(STRPOS(version_raw, '.') = 0, CONCAT(version_raw, '.0'), version_raw),
            ' ', INITCAP(tier),
            IF(subtier IS NULL, '', CONCAT(' ', INITCAP(subtier)))
        )
    END AS model_name,
    CASE
        WHEN LOWER(raw_sku) LIKE '%imagen%' OR LOWER(raw_sku) LIKE '%veo%' THEN
'output'
        WHEN REGEXP_CONTAINS(LOWER(raw_sku), r'cach') THEN
'cached_input'
        WHEN LOWER(raw_sku) LIKE '%output%' THEN
'output'
        WHEN LOWER(raw_sku) LIKE '%input%' THEN
'input'
        ELSE 'other'
    END

```

```

        END AS token_direction,
        CASE
            WHEN LOWER(raw_sku) LIKE '%veo%' THEN IF(LOWER(raw_sku) LIKE '%audio%',
'audio+video', 'video')
            WHEN LOWER(raw_sku) LIKE '%imagen%' THEN 'image'
            WHEN LOWER(raw_sku) LIKE '%audio%' THEN 'audio'
            WHEN LOWER(raw_sku) LIKE '%image%' THEN 'image'
            ELSE 'text'
        END AS modality,
        IF(LOWER(raw_sku) LIKE '%batch%', 'batch', 'online') AS billing_mode,
        REGEXP_CONTAINS(LOWER(raw_sku), r'thinking') AS is_thinking,
        IF(LOWER(raw_sku) LIKE '%imagen%' OR LOWER(raw_sku) LIKE '%veo%', 0,
usage_amount) AS tokens,
        usage_amount,
        usage_unit,
        cost,
        net_cost,
        currency,
        service,
        raw_sku,
        sku_id,
        partition_time,
        (SELECT value FROM UNNEST(labels) WHERE key = 'team' LIMIT 1) AS
label_team,
        (SELECT value FROM UNNEST(labels) WHERE key = 'cost_center' LIMIT 1) AS
label_cost_center,
        (SELECT value FROM UNNEST(labels) WHERE key = 'environment' LIMIT 1) AS
label_environment,
        (SELECT value FROM UNNEST(labels) WHERE key = 'project' LIMIT 1) AS
label_project,
        (SELECT value FROM UNNEST(labels) WHERE key = 'agent' LIMIT 1) AS
label_agent,
        (SELECT value FROM UNNEST(labels) WHERE key = 'adk_agent_name' LIMIT 1) AS
label_google_adk_agent_name --label injected by Google Agent Development Kit.

FROM base;

```

What each column means

Column	Purpose
surface	Gemini API (AI Studio) vs Vertex AI

Column	Purpose
category	llm / image-gen / video-gen — drives whether tokens or images/clips are the natural unit
model_name	Normalized (Gemini 3.0 Pro, Imagen 4 Fast, Veo 3) — folds GA/regional/version-suffix variants
token_direction	input / output / cached_input (LLM); output for media
tokens	Token count — populated only for LLM rows, 0 for media (safe to total)
usage_amount	Raw count — only sum within one category (tokens for LLM, images for Imagen, clips for Veo)
net_cost	Cost adjusted for credits — primary spend metric, safe across all categories
is_thinking	TRUE for thinking-mode output tokens
billing_mode	online vs batch
raw_sku, sku_id	Drill-through to source row for debugging
partition_time	Underlying table's _PARTITIONTIME (ingestion-time TIMESTAMP). Filter on this column in addition to usage_date to enable partition pruning — see Part 3 intro
label_*	Unnested labels refactored as columns. NOTE: The statement above assumes 5 (optional) labels “team”, “cost_center”, “environment”, “agent” and “project” appended to each call to the Gemini API in addition to the default label added by Google ADK. IMP: To successfully achieve cost governance objectives, establishing a shared

Column	Purpose
	label taxonomy across the organization is a crucial step. Refer documentation here on how to attach labels to Gemini and partner model (Anthropic and Mistral) model API calls on the Gemini Enterprise Agent platform.

Part 3 — Sample Queries

Partition pruning tip. The underlying billing-export table is partitioned by ingestion time. The view exposes this as `partition_time`. Adding a filter on `partition_time` alongside the regular `usage_date` filter dramatically reduces bytes scanned (often 10–100×).

Use a small **buffer** (~2 days) because billing rows can land in a partition slightly before/after the usage date due to late corrections:

SQL

```
WHERE partition_time >= TIMESTAMP_SUB(TIMESTAMP(<date_filter>), INTERVAL 2 DAY)
      AND usage_date    >= <date_filter>
```

Q1, Q5, and Q7 below demonstrate the pattern. The other queries omit it for brevity but should be adapted the same way for production use.

Q1: Total Gemini spend by project, current month

SQL

```
SELECT project_id,
       ROUND(SUM(net_cost), 2) AS cost_usd,
       ROUND(SUM(tokens) / 1e6, 2) AS million_tokens
FROM `<PROJECT>.<DATASET>.v_gemini_tokens`
WHERE partition_time >= TIMESTAMP_SUB(TIMESTAMP(DATE_TRUNC(CURRENT_DATE(),
MONTH)), INTERVAL 2 DAY)
      AND usage_date    >= DATE_TRUNC(CURRENT_DATE(), MONTH)
GROUP BY project_id
```



```
ORDER BY cost_usd DESC;
```

Q2: Spend by model, current month — which model drives cost?

SQL

```
SELECT surface, model_name,  
       ROUND(SUM(net_cost), 2) AS cost_usd,  
       ROUND(SUM(tokens) / 1e6, 2) AS million_tokens  
FROM `<PROJECT>.<DATASET>.v_gemini_tokens`  
WHERE usage_date >= DATE_TRUNC(CURRENT_DATE(), MONTH)  
      AND category = 'llm'  
GROUP BY surface, model_name  
ORDER BY cost_usd DESC;
```

Q3: Input vs Output vs Cached split per model

SQL

```
SELECT model_name, token_direction,  
       ROUND(SUM(tokens) / 1e6, 2) AS million_tokens,  
       ROUND(SUM(net_cost), 2) AS cost_usd  
FROM `<PROJECT>.<DATASET>.v_gemini_tokens`  
WHERE usage_date >= DATE_SUB(CURRENT_DATE(), INTERVAL 30 DAY)  
      AND category = 'llm'  
GROUP BY model_name, token_direction  
ORDER BY model_name, token_direction;
```

Q4: Month-over-month trend (last 6 months)

SQL

```
SELECT DATE_TRUNC(usage_date, MONTH) AS month,  
       project_id,  
       ROUND(SUM(net_cost), 2) AS cost_usd  
FROM `<PROJECT>.<DATASET>.v_gemini_tokens`  
WHERE usage_date >= DATE_SUB(DATE_TRUNC(CURRENT_DATE(), MONTH), INTERVAL 5 MONTH)  
GROUP BY month, project_id  
ORDER BY month DESC, cost_usd DESC;
```

Q5: MTD vs prior 2 MTDs (same day-of-month window) per project

Useful for "is anyone trending higher than usual?"

SQL

```
WITH today AS (SELECT CURRENT_DATE() AS d)
SELECT
  project_id,
  ROUND(SUM(IF(DATE_TRUNC(usage_date, MONTH) = DATE_TRUNC(today.d, MONTH)
    AND usage_date <= today.d, net_cost, 0)), 2) AS mtd_current,
  ROUND(SUM(IF(DATE_TRUNC(usage_date, MONTH) = DATE_TRUNC(DATE_SUB(today.d,
INTERVAL 1 MONTH), MONTH)
    AND usage_date <= DATE_SUB(today.d, INTERVAL 1 MONTH), net_cost,
0)), 2) AS mtd_prev_1,
  ROUND(SUM(IF(DATE_TRUNC(usage_date, MONTH) = DATE_TRUNC(DATE_SUB(today.d,
INTERVAL 2 MONTH), MONTH)
    AND usage_date <= DATE_SUB(today.d, INTERVAL 2 MONTH), net_cost,
0)), 2) AS mtd_prev_2
FROM `<PROJECT>.<DATASET>.v_gemini_tokens`, today
WHERE partition_time >= TIMESTAMP_SUB(
  TIMESTAMP(DATE_TRUNC(DATE_SUB(CURRENT_DATE(), INTERVAL 2 MONTH), MONTH)),
  INTERVAL 2 DAY)
GROUP BY project_id
ORDER BY mtd_current DESC;
```

Q6: Cost per 1M tokens (unit economics) by model

SQL

```
SELECT model_name, token_direction,
  ROUND(SUM(net_cost) / NULLIF(SUM(tokens), 0) * 1e6, 4) AS
usd_per_million_tokens,
  ROUND(SUM(tokens) / 1e6, 2) AS million_tokens
FROM `<PROJECT>.<DATASET>.v_gemini_tokens`
WHERE usage_date >= DATE_SUB(CURRENT_DATE(), INTERVAL 30 DAY)
  AND category = 'llm'
  AND tokens > 0
GROUP BY model_name, token_direction
ORDER BY model_name, token_direction;
```

Q7: Daily spike detection — projects whose daily spend > 3× their 30-day average

SQL

```
WITH daily AS (
  SELECT project_id, usage_date, SUM(net_cost) AS daily_cost
  FROM `<PROJECT>.<DATASET>.v_gemini_tokens`
  WHERE partition_time >= TIMESTAMP_SUB(TIMESTAMP(DATE_SUB(CURRENT_DATE(),
INTERVAL 31 DAY)), INTERVAL 2 DAY)
  AND usage_date >= DATE_SUB(CURRENT_DATE(), INTERVAL 31 DAY)
  GROUP BY project_id, usage_date
),
```

```

stats AS (
  SELECT project_id, AVG(daily_cost) AS avg_30d
  FROM daily
  WHERE usage_date < CURRENT_DATE()
  GROUP BY project_id
)
SELECT d.usage_date, d.project_id, ROUND(d.daily_cost, 2) AS daily_cost,
       ROUND(s.avg_30d, 2) AS avg_30d,
       ROUND(d.daily_cost / NULLIF(s.avg_30d, 0), 1) AS multiple_of_avg
FROM daily d
JOIN stats s USING (project_id)
WHERE d.daily_cost > 3 * s.avg_30d
      AND d.daily_cost > 1 -- ignore noise under $1
ORDER BY d.usage_date DESC, multiple_of_avg DESC;

```

Q8: Cached-input savings — is prompt caching paying off?

```

SQL
SELECT model_name,
       ROUND(SUM(IF(token_direction = 'input', net_cost, 0)), 2) AS
uncached_input_cost,
       ROUND(SUM(IF(token_direction = 'cached_input', net_cost, 0)), 2) AS
cached_input_cost,
       ROUND(SUM(IF(token_direction = 'input', tokens, 0)) / 1e6, 2) AS
uncached_input_mtok,
       ROUND(SUM(IF(token_direction = 'cached_input', tokens, 0)) / 1e6, 2) AS
cached_input_mtok
FROM `<PROJECT>.<DATASET>.v_gemini_tokens`
WHERE usage_date >= DATE_SUB(CURRENT_DATE(), INTERVAL 30 DAY)
      AND category = 'llm'
GROUP BY model_name
HAVING cached_input_mtok > 0
ORDER BY cached_input_cost DESC;

```

Q9: Media generation — Imagen and Veo volume

```

SQL
SELECT model_name, category, modality,
       SUM(usage_amount) AS units_generated,
       ROUND(SUM(net_cost), 2) AS cost_usd
FROM `<PROJECT>.<DATASET>.v_gemini_tokens`
WHERE usage_date >= DATE_SUB(CURRENT_DATE(), INTERVAL 30 DAY)
      AND category IN ('image-gen', 'video-gen')
GROUP BY model_name, category, modality
ORDER BY cost_usd DESC;

```

Q10: Token Cost incurred by ADK agent “ABC”

SQL

```
SELECT project_id, usage_date, sum(net_cost) FROM  
`<PROJECT>.<DATASET>.v_gemini_tokens` WHERE label_google_adk_agent_name  
='ABC' GROUP BY 1,2
```

Part 4 — Maintenance & Gotchas

When Google adds new Gemini SKUs

The view is filtered by SKU naming patterns. New models follow Google's existing naming conventions, so most slot in automatically. Verify monthly with:

SQL

```
SELECT DISTINCT raw_sku  
FROM `<PROJECT>.<DATASET>.v_gemini_tokens`  
WHERE model_name LIKE 'UNPARSED:%';
```

If any rows return, extend the regex in the `model_name` CASE.

When new categories appear (e.g., new media-gen products)

The current view captures Imagen and Veo. To add a different/new product (e.g., Lyria for music), update the WHERE clause and add a `category` branch.

`usage.unit` is always `requests`

Usage.unit in the raw table reports `requests` for every token SKU. The amount itself is the token count. The view's `tokens` column abstracts this.

Billing export latency

Detailed export refreshes a few times per day, typically 4–6 hour lag. Same-day spend is incomplete until ~24 hours after.

Granting dashboard access

SQL

```
GRANT `roles/bigquery.dataViewer` ON SCHEMA `<PROJECT>.<DATASET>`  
TO 'group:gemini-cost-viewers@example.com';
```

Model Garden (Llama, Deepseek, Gemma) is excluded

The view filters those out (`NOT LIKE '%model garden%'`). To track them too, add an `OR` branch to the `WHERE` and a category like `model-garden-llm`.

AI Studio API-key usage attribution

Gemini API (AI Studio) projects show up by the project that owns the API key. If usage shows under `org-level (unattributed)`, the API key was created outside any project (rare) — investigate the responsible team.

Part 5 — Build a Data Studio Dashboard

Once the data is available in BQ we can use [DataStudio](#) to create dashboards with bar charts, pivot tables, and shared filters — like the screenshot below. Steps to create the dashboard are summarized; refer to Data Studio docs for UI-level detail.

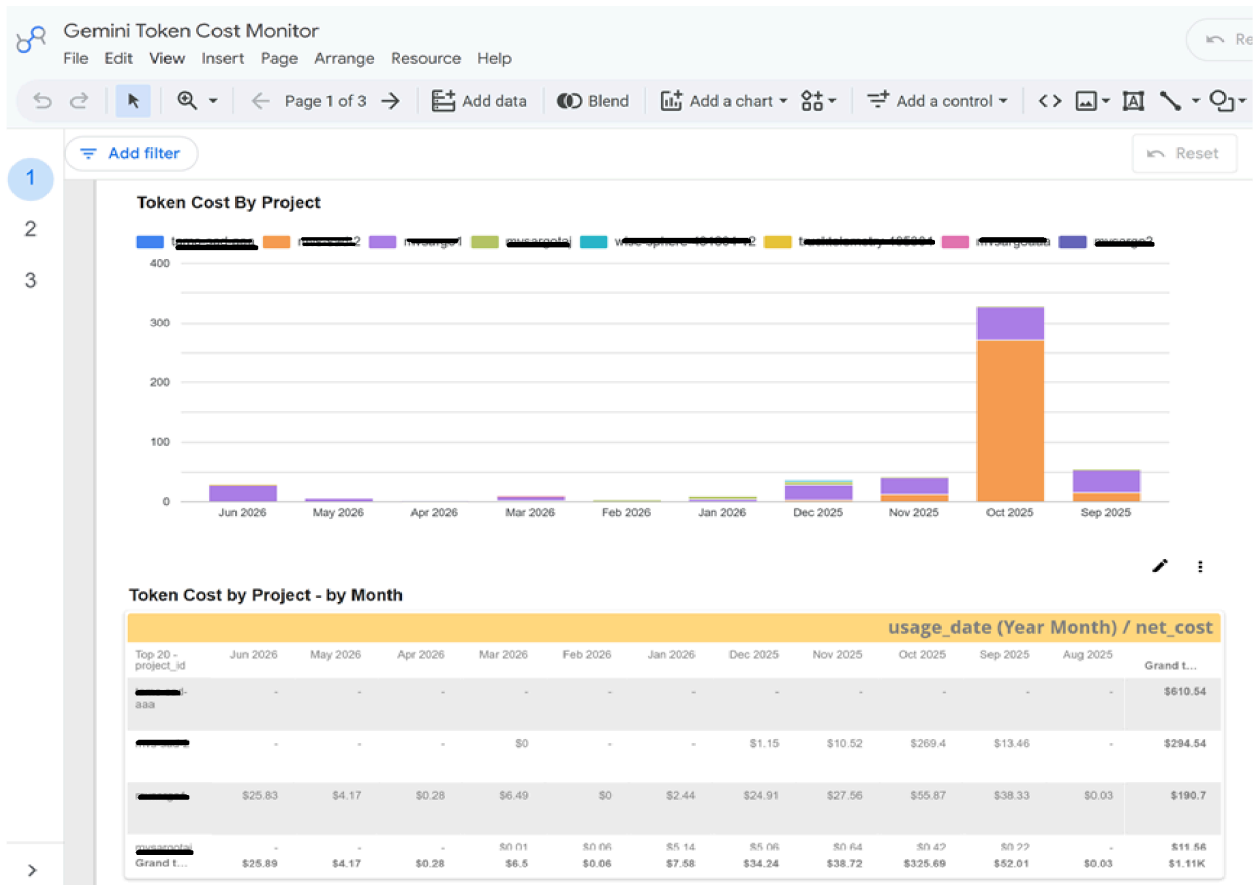


Fig: Sample dashboard showing month wise token consumption cost across projects

Steps to build a simple dashboard are detailed below.

Target layout:

- Top: a **model_name** dropdown filter and a date range picker
- Middle: a **monthly bar chart** of net_cost by project (**Token Cost by Project**)
- Bottom: a **pivot table** with projects as rows, months as columns, net_cost as cells, with row/column grand totals (**Token Cost by Project – by Month**)

Step 1 — Connect the data source

1. Open <https://datastudio.google.com> → **Create** → **Data Source** → **BigQuery**.
2. Pick **<PROJECT>** → **<DATASET>** → **v_gemini_tokens** → **Connect**.
3. In the field editor, set **net_cost** to **Type: Currency (USD)**, default aggregation **Sum**. This makes every chart render \$ automatically.

Step 2 — Add shared filter controls (top of the page)

Use **Add a control**:

- **Date range control** — default **Last 12 months**.
- **Drop-down list** → Control field: **model_name** (multi-select).
- Optionally add the same for **project_id** and **surface**.

These controls apply to any chart whose **Date range dimension** is set to **usage_date**.

Step 3 — Build the bar chart ("Token Cost by Project")

- Chart type: **Column chart** (stacked).
- **Dimension:** **usage_date** → granularity **Year Month**.
- **Breakdown dimension:** **project_id**.
- **Metric:** **net_cost** (Sum).
- **Sort:** **usage_date Ascending** (for chronological left→right).
- **Style** → **Show data labels:** ON, decimal places **0**.

Step 4 — Build the pivot table ("Token Cost by Project — by Month")

- Chart type: **Pivot table** (plain — *not* the heatmap variant).
- **Row dimension:** **project_id**.
- **Column dimension:** **usage_date** → granularity **Year Month**.
- **Metric:** **net_cost** (Sum).
- **Style** → **Totals:** turn ON both **Row totals** and **Column totals**.
- Sort rows by row total descending; sort columns by **usage_date** (asc or desc, your preference).

Step 5 — Wire the date control

For both charts, in Setup confirm **Date range dimension = partition_time(Date)** (the original Date field, green calendar icon — not **(Number)** or **(ISO Week)** variants). If empty, the date picker won't filter that chart.

Optional — Executive page

Add a second page with: 4 scorecards (Total spend, Total tokens, Active projects, \$/1M tokens), a project × model pivot with conditional formatting, and an MTD-vs-prior-MTD comparison table.

For MTD comparison, add these calculated fields at the data source level:

```
None
-- mtd_current
CASE WHEN DATETIME_TRUNC(usage_date, MONTH) = DATETIME_TRUNC(TODAY(), MONTH)
      AND usage_date <= TODAY()
      THEN net_cost ELSE 0 END
```

```
-- mtd_prev_1
CASE WHEN DATETIME_TRUNC(usage_date, MONTH) = DATETIME_TRUNC(DATETIME_SUB(TODAY(),
INTERVAL 1 MONTH), MONTH)
      AND usage_date <= DATETIME_SUB(TODAY(), INTERVAL 1 MONTH)
      THEN net_cost ELSE 0 END

-- mtd_prev_2 (use INTERVAL 2 MONTH in both places)
```

Use these as side-by-side metrics in a table dimensioned by `project_id`. Add a chart-level calculated metric $(\text{SUM}(\text{mtd_current}) - \text{SUM}(\text{mtd_prev_1})) / \text{SUM}(\text{mtd_prev_1})$ formatted as Percent for the MoM $\Delta\%$ column.

Agent cost attribution

As discussed in the section above Google ADK automatically adds a label to Gemini API calls with value set to the agent name as defined in code. Additionally it is recommended to add custom labels to API calls (whether invoked through agents or otherwise). These labels are exported to BigQuery along with the billing information allowing us to build detailed cost attribution reports.

A sample report below shows model consumption across 3 agents in a sankey + pivot visualization

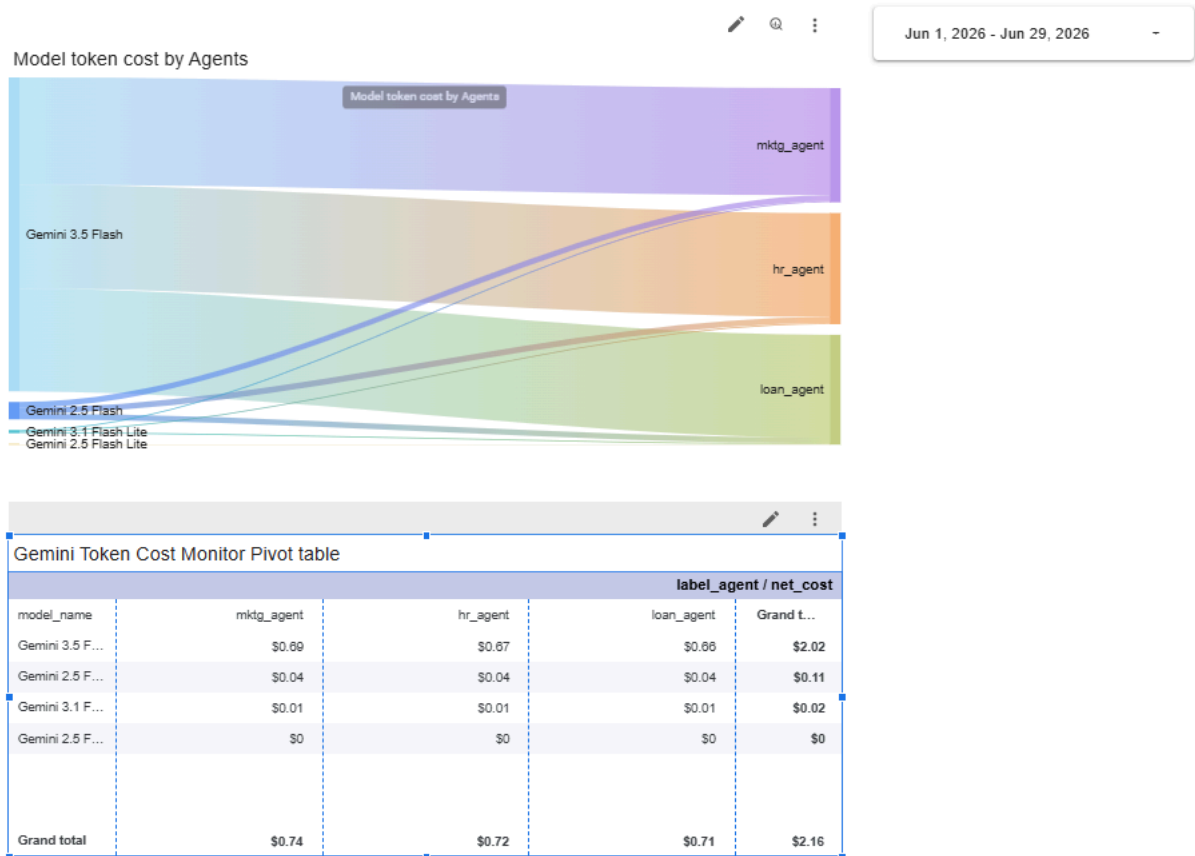


Fig: Data studio dashboard tracking agent wise token costs

Part 6 — Analyze in Google Sheets (Connected Sheets)

For users more comfortable in Google Sheets than in Data Studio, the **Connected Sheets** feature lets you point a Google Sheet directly at the `v_gemini_tokens` view and build pivots / charts / formulas on top — without exporting data or losing freshness. The pivot below was built entirely in a connected sheet:

- Rows: `project_id/model_name`
- Columns: `year_month`
- Values: `SUM(net_cost)`
- Grand Total row at the bottom

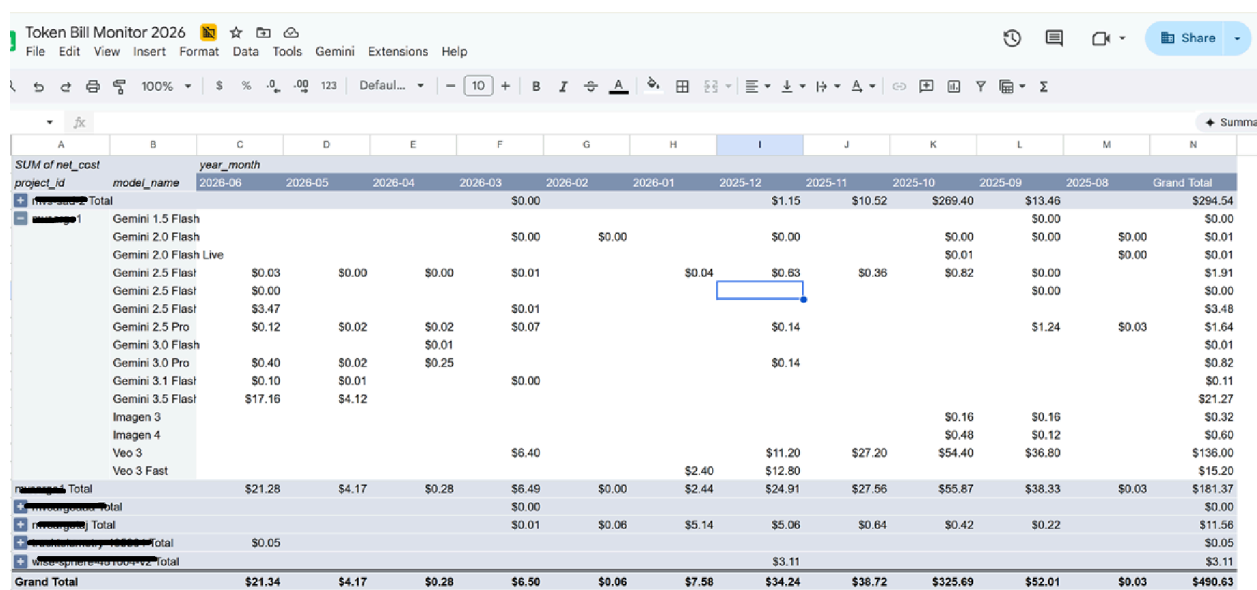


Fig: Sample Google sheets pivot connected to billing 'view' in BigQuery

When to use this vs. Data Studio

Connected Sheets	Data Studio
Ad-hoc analysis, one-off questions	Always-on dashboards
Sheets-native pivots, formulas, charts	Polished filter controls, scorecards
Shareable as a working file	Shareable as a viewer link
Refresh on demand or scheduled	Auto-refresh on load

Step 1 — Create the connected sheet

1. Open a new Google Sheet → **Data** → **Data connectors** → **Connect to BigQuery**.
2. Pick the project (e.g., **billAnalytics**) → dataset **<DATASET>** → table **v_gemini_tokens** → **Connect**.
3. The sheet shows a preview of the view's first ~500 rows; the full dataset stays in BigQuery and is queried on demand.

Costs: each refresh runs a BigQuery query and is billed to the connected project under standard BQ on-demand pricing. The view is small — typical refreshes cost cents.

Step 2 — (Optional) Add a `year_month` calculated column

The view exposes `usage_date` as a daily date. To pivot by month, click **Calculated column** in the toolbar and add:

```
None  
=TEXT(usage_date, "YYYY-MM")
```

Name the column `year_month`. (Alternative: skip this step and use the **Group by** → **Year-Month** option directly inside the pivot editor.)

Step 3 — Build the pivot table

1. Click **Pivot table** in the toolbar (next to **Chart**).
2. Choose where to insert (new sheet recommended).
3. Configure in the pivot editor:
 - **Rows:** `project_id`
 - **Columns:** `year_month` (sort descending for newest months on the left)
 - **Values:** `net_cost`, summarized by **SUM**, formatted as **Currency**
4. Grand Total appears automatically; toggle row/column totals if you want both.

Step 4 — Refresh

- **Manual:** click **Refresh** at the top of the pivot or use **Refresh options** → **Refresh all**.
- **Scheduled:** **Schedule refresh** lets you set a daily/hourly cadence so the sheet stays current without manual action.

Step 5 — (Recommended for production) Partition-prune via a `Config` parameter

The Step 1 path connects to the view directly, so every refresh queries across all partitions of the underlying billing-export table. For datasets with significant history, switch to a **custom query** with a `@START_DATE` parameter bound to a cell on a `Config` tab. The query then filters on `partition_time` and sheet users can change the lookback window by editing one cell — no SQL edits needed.

Setup:

1. Add a tab named `Config`. Pick a cell (e.g., `Config!C3`) and put the earliest date to load (e.g., `3/31/2026`). Label it "Start Date" in an adjacent cell.
2. In **Data** → **Data connectors** → **BigQuery**, switch from **Table** to **Custom query** and paste:

SQL

SELECT

```
usage_date, project_id, project_name, surface, category, model_name,
token_direction, modality, billing_mode, is_thinking,
tokens, usage_amount, usage_unit, cost, net_cost, currency,
service, raw_sku, sku_id, partition_time,
FORMAT_DATE('%Y-%m', usage_date) AS year_month,
CONCAT(CAST(EXTRACT(YEAR FROM usage_date) AS STRING), '-Q',
        CAST(EXTRACT(QUARTER FROM usage_date) AS STRING)) AS year_quarter,
FORMAT_DATE('%Y-Wk %V', usage_date) AS week_of_year
FROM `<PROJECT>.<DATASET>.v_gemini_tokens`
WHERE EXTRACT(DATE FROM partition_time) >= @START_DATE
ORDER BY usage_date DESC
```

3. In the BigQuery editor, open the **Parameters** panel → **Add** → name **START_DATE** → cell reference **Config!C3** → **Connect**.

BigQuery query editor

```
11  modality,
12  billing_mode,
13  is_thinking,
14  tokens,
15  usage_amount,
16  usage_unit,
17  cost,
18  net_cost,
19  currency,
20  service,
21  raw_sku,
22  sku_id,
23  CONCAT(CAST(EXTRACT(YEAR
24  FROM
25      usage_date) AS STRING), '-Q', CAST(EXTRACT(QUARTER
26  FROM
27      usage_date) AS STRING)) AS year_quarter,
28  FORMAT_DATE('%Y-%m', usage_date) AS year_month,
29  FORMAT_DATE('%Y-Wk %V', usage_date) AS week_of_year,
30  partition_time
31  FROM
32  `mvsargol`.`MVSBillingDS`.`v_gemini_tokens`
33  WHERE
34  EXTRACT(DATE FROM partition_time)>@START_DATE
35  ORDER BY
36  usage_date DESC
37  LIMIT 10000
38
```

The query and resulting data are visible to all users with access to the spreadsheet.
Learn more about [BigQuery data in Connected Sheets](#).

Preview results

Connect

Fig: Configuring the BQ connection in Google Sheets to filter dates

Every refresh now scans only partitions on or after the Start Date. Users widen or narrow the load by editing **Config!C3** and hitting refresh.

Stricter correctness (optional) — to match the partition-pruning pattern from Part 3 exactly (with a 2-day buffer for late-arriving rows), replace the **WHERE** clause with:

SQL

```
WHERE partition_time >= TIMESTAMP_SUB(TIMESTAMP(@START_DATE), INTERVAL 2 DAY)
AND usage_date >= @START_DATE
```

Note: the SQL above also adds **year_month**, **year_quarter**, and **week_of_year** directly — so when using this approach you can skip Step 2's in-sheet calculated column.

Permissions for sheet users

End users who only consume the sheet need **roles/bigquery.dataViewer** on the dataset *and* **roles/bigquery.jobUser** on the project that runs the query (Connected Sheets needs both: data access + the ability to start a query job).

The sheet can be scheduled to update at a fix times per day using the “**Schedule refresh**” feature so that the data in the sheets is automatically updated and need not be updated manually,

Part 7 — Build a "Token_Cost_helper" Data Agent (Conversational Analytics)

For admins and finance partners who'd rather **ask questions in English** than write SQL or build pivots, BigQuery's **Conversational Analytics** lets you stand up a chat-style data agent pointed at the same view. The agent generates SQL, runs it, and returns tables, charts, and narrative summaries.

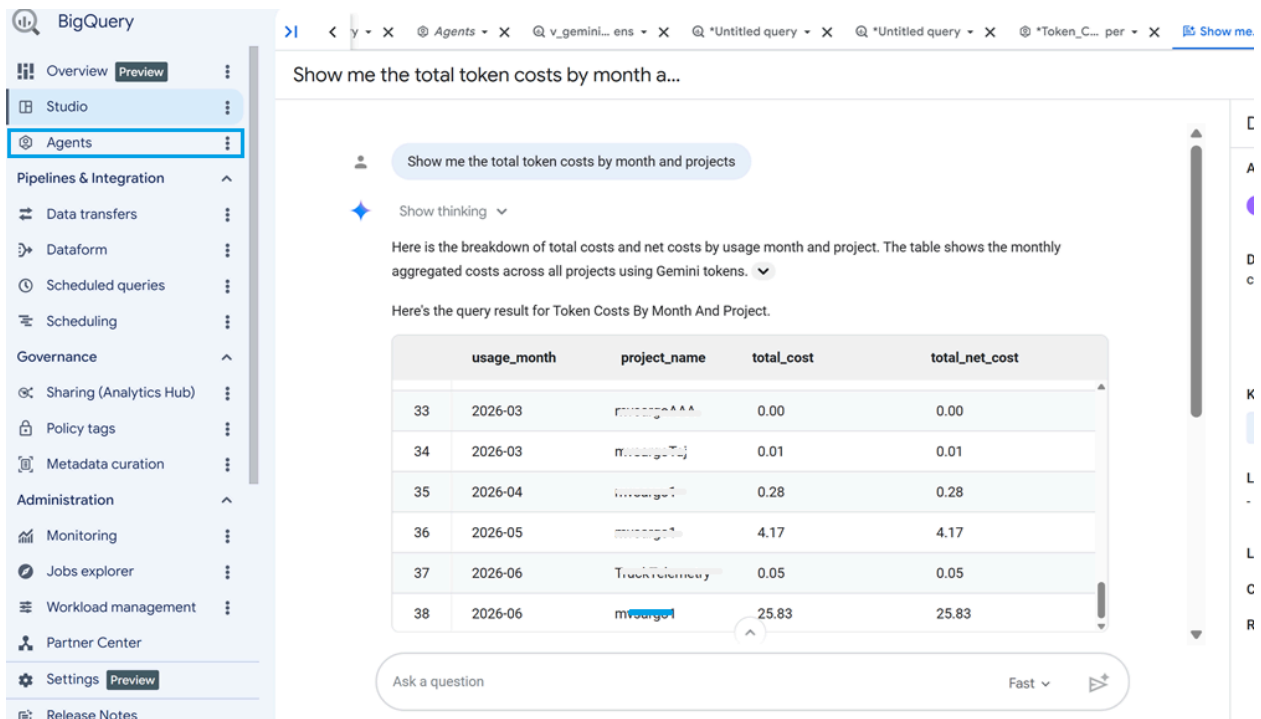


Fig: Sample Data Agent in BQ answering billing questions.

Outcome: an agent named `Token_Cost_helper` answering questions like "What's the MTD token bill so far?", "Which project is trending higher this month?", "Show me cost by model for May vs June."

Prerequisites

- BigQuery Conversational Analytics API enabled on the host project.
- IAM: `roles/geminiataanalytics.dataAgentUser` (use the agent), `roles/geminiataanalytics.dataAgentEditor` (build/edit it).
- The agent's end users still need `bigquery.dataViewer` on the dataset — the agent runs queries with the *user's* identity, not its own.

Step 1 — Create the agent

1. In the BigQuery console, open the left nav → **Agents** → **+ Create agent**.
2. Fill the editor:
 - **Agent name:** `Token_Cost_helper`
 - **Agent description:** *Helps answer token consumption and cost queries from the GCP Billing dataset.*
 - **Region:** keep the same region as your billing dataset.

Step 2 — Attach the knowledge source

Under **Knowledge sources** → **Add source**, pick **BigQuery table** and select:

None

`<PROJECT>.<DATASET>.v_gemini_tokens`

Click **Customize** on the source to:

- Give the agent **column descriptions** (e.g., "`token_direction` — one of input, output, cached_input"). Good descriptions dramatically improve generated-SQL accuracy.
- Mark `net_cost`, `tokens`, `usage_amount` as numeric measures.
- For `partition_time`, the description matters most: clarify that it's the **ingestion-time TIMESTAMP exposed for partition pruning**, and that user-facing date filters should still use `usage_date` semantically. Without this hint the agent may filter on `partition_time` for date questions, returning slightly wrong results around month boundaries.

You can attach multiple sources; for this agent the single view is enough since it already normalizes everything.

Step 3 — Write system instructions

Keep instructions short and explicit. The screenshot uses:

None

`Only Answer queries on the project wise token consumption and cost.
Allow drill down queries by usage month and model type.`

Useful additions for accuracy:

- *Always use `net_cost` (not `cost`) for spend totals.*
- *For any time-bounded question, include a `partition_time` filter alongside the `usage_date` filter to prune partitions — use a 2-day buffer: `WHERE partition_time >= TIMESTAMP_SUB(TIMESTAMP(<date_filter>), INTERVAL 2 DAY) AND usage_date >= <date_filter>`.*
- *For LLM token volumes, filter `category` = '`llm`'. For media generation use `usage_amount` filtered to `image-gen` or `video-gen`.*
- *Treat `cached_input` as a separate direction, not part of `input`.*
- *Default time window: last 30 days unless the user specifies otherwise.*

Step 4 — Seed verified queries (most important step)

Verified queries are trusted question→SQL pairs that anchor the agent's behavior for common questions. The agent prefers these over freshly-generated SQL when a similar question comes in. Add 5–10 of these — quality compounds.

Suggested verified queries (use the SQL from Part 3 of Section 2):

What's the total Gemini spent by project this month?

Which model is driving the most cost?

Show input vs output vs cached tokens per model.

What was the month-over-month trend for the last 6 months?

What's the MTD spend vs the prior two MTDs?

What's the cost per million tokens by model?

Which projects spent more than 3× their average yesterday?

How much are we saving with cached input?

Add each via **Verified queries** → **Add** and paste the SQL.

Step 5 — Test in the Preview pane

The right-side **Preview** chat lets you try questions before publishing:

- *"What's the MTD token bill so far?"*
- *"Compare Gemini 2.5 Pro vs Gemini 3.0 Pro by spend this quarter."*
- *"Which project's cost is trending up the most?"*

Look for:

- **SQL it generated** (expand "Show thinking") — sanity-check the WHERE clause and joins.
- **Hallucinated column names** — if it invents columns, your column descriptions need to be more explicit, or add a verified query covering that intent.
- **Chart choice** — agent picks bar/line/table automatically. Adjust by asking *"show as a bar chart"*.

Step 6 — Publish and share

- Click **Publish** when the agent answers reliably.
- **Share** → add user emails or groups with the **User** role.
- Users access the agent via the BigQuery console (**Agents** → **Token_Cost_helper**) or via the **Create conversation** link for a chat-only view.

Operational notes

- **Cost:** each question fires one or more BigQuery queries plus a Gemini call. On a small view like this, expect well under \$0.01 per question.
- **Refresh:** the agent reads the view live — no separate refresh needed. New data lands as the billing export updates.
- **Limits:** Conversational Analytics works best on **denormalized, well-described** sources. The `v_gemini_tokens` view is already shaped for this — flat, single-table, parsed columns, no joins required.

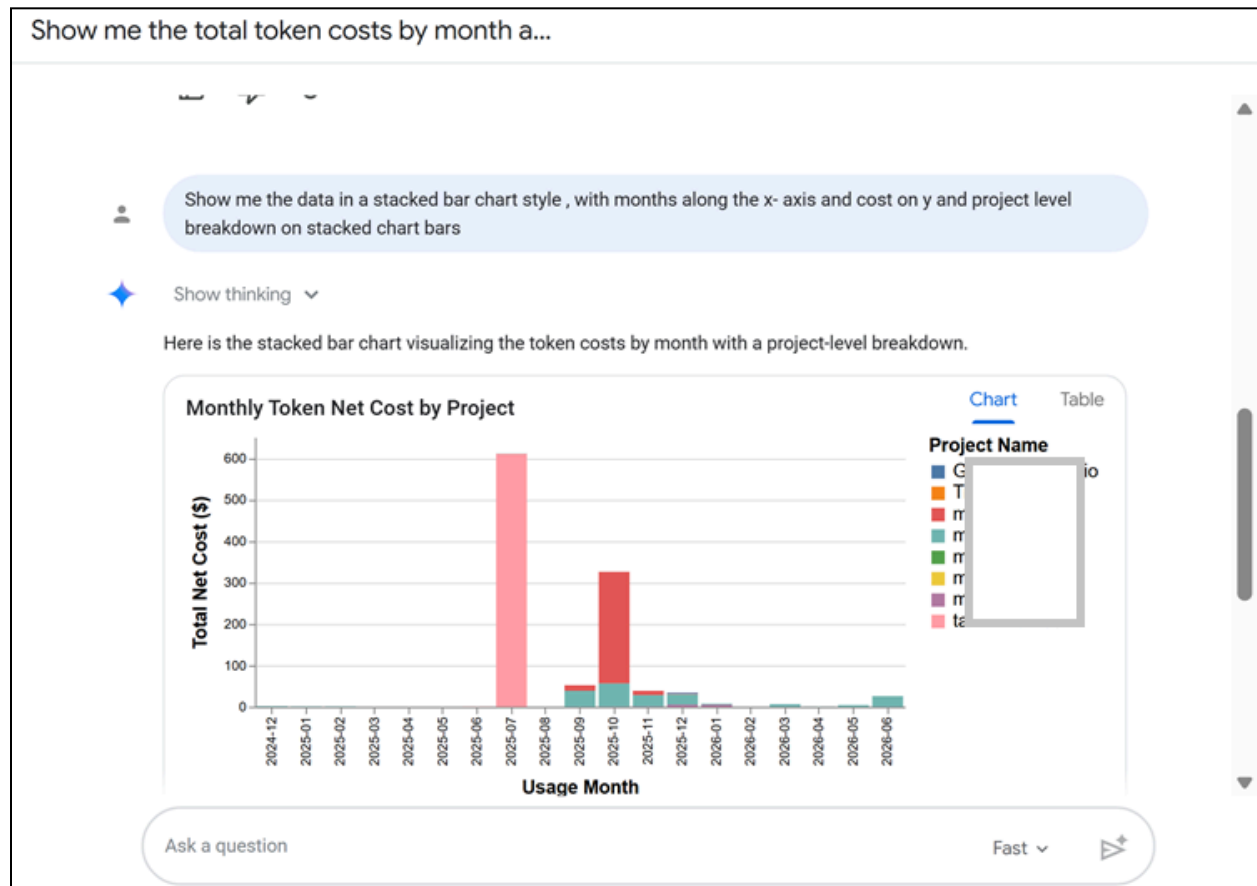


Fig:Data Agent generating charts.

The link to the chat agent can directly be shared with other users (having relevant permissions) using the “Share” feature

When to use this vs. the other tools

Use the “agent” for	Use Data Studio for	Use Connected Sheets for
Ad-hoc questions in English	Always-on dashboards	Ad-hoc pivots & spreadsheets
Investigating anomalies interactively	Exec/team visibility	Sheets-native modeling
Quick "what if" follow-ups in a chat	Period reports	Sharing a working file

What Next?

As your adoption of agentic AI grows, your token consumption will come in via various routes viz. direct Gemini API calls in applications, API calls via custom AI agents, API calls via AI-assisted tools viz. Antigravity, Claude Code. This document helps you set up the cost monitoring tools to allow you deep visibility.

Once token consumption patterns are known organizations can safely employ techniques to manage costs by identifying teams consuming most tokens and working with them cost control techniques viz. **correct model selection** (viz. choosing flash vs. pro models where appropriate, gemini-3.5-flash vs gemini-3.1-pro), deeper **context engineering** to improve accuracy of outputs by **tuning prompts**, so that they generate more accurate content the first time, **using ‘skills’** while developing agents etc.

At the time of writing Google Cloud has announced more cost governance features such as automated [spend caps and fin-ops explainability](#) agent to further enhance cost governance feature set.